Лабораторная работа №4

Модули, исключения и работа с файлами

1. Цель работы

Изучить разделение приложения на модули, собственные исключения, обработку ошибок, сериализацию объектов и хранение данных в JSON.

2. Постановка индивидуального задания

Вариант 12 — «Учебные курсы». Реализовать многофайловое приложение с хранением данных.

Программа должна:

– Создать учёт курсов с названием, преподавателем, длительностью, ценой и списком студентов.

– Реализовать создание, поиск, запись/отчисление студентов и среднее число студентов на курс.

– Организовать обработку ошибок и хранение в файле JSON.

3. Краткие теоретические сведения

Модуль — отдельный файл Python. Разделение моделей (models.py), операций (services.py), хранения (storage.py), исключений и пользовательского меню делает программу понятнее и упрощает изменения.

Для предметной ошибки используется пользовательский класс CourseError. Блок try/except перехватывает ожидаемые ошибки и сообщает о них пользователю, предотвращая аварийное завершение меню.

JSON — текстовый формат хранения структурированных данных. json.dumps(..., ensure_ascii=False) сериализует список словарей; json.loads() выполняет обратное преобразование. В реализации объект Course восстанавливается посредством Course(**item).

4. Исходный код программы

Файл exceptions.py

```
class CourseError(Exception):
    pass
```

Файл models.py

```
from exceptions import CourseError


class Course:
    def __init__(self, name, teacher, hours, price, students=None):
        if not name or not teacher or hours <= 0 or price < 0:
            raise CourseError("Неверные данные курса")
        self.name = name
        self.teacher = teacher
        self.hours = hours
        self.price = price
        self.students = students if students is not None else []

    def __str__(self):
        return f"{self.name}: {self.teacher}, {self.hours} ч, {self.price} руб., {self.students}"
```

Файл services.py

```
from exceptions import CourseError


class CourseService:
    def __init__(self, courses):
        self.courses = courses

    def create(self, course):
        if any(c.name == course.name for c in self.courses):
            raise CourseError("Такой курс уже есть")
        self.courses.append(course)

    def find(self, name):
        for course in self.courses:
            if course.name == name:
                return course
        raise CourseError("Курс не найден")

    def add_student(self, name, student):
        course = self.find(name)
        if not student or student in course.students:
            raise CourseError("Пустое имя или студент уже записан")
        course.students.append(student)

    def remove_student(self, name, student):
        course = self.find(name)
        if student not in course.students:
            raise CourseError("Студент не найден")
        course.students.remove(student)
```

```
    def average_load(self):
        if not self.courses:
            return 0
        return sum(len(c.students) for c in self.courses) / len(self.courses)
```

Файл storage.py

```
import json
from pathlib import Path
from models import Course

FILE = Path(__file__).with_name("courses.json")


def save(courses):
    data = [vars(course) for course in courses]
    FILE.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")


def load():
    if not FILE.exists():
        return []
    data = json.loads(FILE.read_text(encoding="utf-8"))
    return [Course(**item) for item in data]
```

Файл main.py

```
from models import Course
from services import CourseService
from exceptions import CourseError
from storage import load, save


def main():
    try:
        service = CourseService(load())
    except (OSError, ValueError, TypeError, CourseError) as error:
        print("Не удалось загрузить данные:", error)
        return

    while True:
        print("\n1 Создать курс; 2 Добавить студента; 3 Удалить студента")
        print("4 Найти курс; 5 Средняя загрузка; 6 Все курсы; 7 Сохранить; 0 Выход")
        choice = input("Действие: ")
        try:
            if choice == "1":
                course = Course(input("Название: "), input("Преподаватель: "),
                                int(input("Часы: ")), float(input("Стоимость: ")))
                service.create(course)
            elif choice == "2":
                service.add_student(input("Курс: "), input("Студент: "))
            elif choice == "3":
                service.remove_student(input("Курс: "), input("Студент: "))
            elif choice == "4":
                print(service.find(input("Курс: ")))
            elif choice == "5":
```

```
            print("Среднее количество студентов на курс:", service.average_load())
        elif choice == "6":
            for course in service.courses:
                print(course)
        elif choice == "7":
            save(service.courses)
            print("Сохранено")
        elif choice == "0":
            save(service.courses)
            break
        else:
            print("Неизвестное действие")
    except (CourseError, ValueError, OSError) as error:
        print("Ошибка:", error)


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 4.1 показаны создание и поиск учебного курса, добавление студента и сохранение данных в JSON.

```
python main.py
Создан курс: Алгоритмы, Иванов, 36 ч, 4800 руб.
Записан студент: Олег
Алгоритмы: Иванов, 36 ч, 4800.0 руб., ['Олег']
Среднее количество студентов на курс: 1.0
Python: Иванов, 40 ч, 5000.0 руб., ['Сергей']
Алгоритмы: Иванов, 36 ч, 4800.0 руб., ['Олег']
Сохранено
```

Рисунок 4.1 – Результаты работы программы учёта курсов

6. Вывод

Создано многофайловое приложение учёта курсов с собственной ошибкой, отделённой бизнес-логикой и JSON-хранилищем. Проверены создание и поиск курсов, запись студента и сохранение данных. Поддержка CSV остаётся нереализованной.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	